

Mảng hậu tố

Nguồn gốc: *Suffix array*

data-structure/suffix-array.pdf

1 Thuật toán nhân đôi

Ý tưởng chính là trước hết sắp xếp từng ký tự đơn lẻ. Sau đó, dựa trên kết quả sắp xếp từng ký tự, ta dịch vị trí để thu được kết quả sắp xếp của ký tự đơn lẻ kế tiếp; sắp xếp lại sẽ thu được thứ tự của các xâu con gồm hai ký tự. Tiếp tục dịch vị trí rồi trộn kết quả sắp xếp để thu được thứ tự của các xâu con gồm bốn ký tự, v.v. Vì vậy, độ dài xâu con cần xác định thứ tự tăng theo hàm mũ. Kết hợp với sắp xếp cơ số, mỗi lượt có độ phức tạp $O(n)$, nên tổng độ phức tạp thời gian là $O(n \log n)$.

```
int wa[maxn],wb[maxn],wv[maxn],ws[maxn];
int cmp(int *r,int a,int b,int l) {
    return r[a]==r[b]&&r[a+l]==r[b+l];
}
void da(int *r,int *sa,int n,int m) {
    int i,j,p,*x=wa,*y=wb,*t;
    for(i=0;i<m;i++) ws[i]=0;
    for(i=0;i<n;i++) ws[x[i]=r[i]]++;
    for(i=1;i<m;i++) ws[i]+=ws[i-1];
    for(i=n-1;i>=0;i--) sa[--ws[x[i]]]=i;
    for(j=1,p=1;p<n;j*=2,m=p)
    {
        for(p=0,i=n-j;i<n;i++) y[p++]=i;
        for(i=0;i<n;i++) if(sa[i]>=j) y[p++]=sa[i]-j;
        for(i=0;i<n;i++) wv[i]=x[y[i]];
        for(i=0;i<m;i++) ws[i]=0;
        for(i=0;i<n;i++) ws[wv[i]]++;
        for(i=1;i<m;i++) ws[i]+=ws[i-1];
        for(i=n-1;i>=0;i--) sa[--ws[wv[i]]]=y[i];
        for(t=x,x=y,y=t,p=1,x[sa[0]]=0,i=1;i<n;i++)
            x[sa[i]]=cmp(y,sa[i-1],sa[i],j)?p-1:p++;
    }
    return;
}
```

1. Bốn mảng wa , wb , wv , ws là các mảng phụ trợ. Trong quá trình chạy chương trình, chúng có các ý nghĩa khác nhau; các mục sau sẽ giải thích cụ thể.
2. Hàm cmp ở dòng 2–4 dùng để so sánh xem hai khóa có giống nhau hay không. Khi ban đầu so sánh thứ tự từng ký tự, khóa là mã ASCII của ký tự; về sau, khi đồng thời so sánh thứ tự của 2^k ký tự với $k \geq 1$, khóa là thứ hạng của 2^{k-1} ký tự. Nếu hai khóa giống nhau, tức hai ký tự hoặc hai xâu con

hoàn toàn giống nhau, hàm trả về 1; nếu không, hàm trả về 0. Cụ thể, khi so sánh các xâu độ dài 2^k , giá trị l là 2^{k-1} , nghĩa là nửa trước có cùng thứ hạng và nửa sau cũng có cùng thứ hạng.

3. Trong khai báo hàm, r là xâu ban đầu, sa là mảng hậu tố, n là độ dài xâu ban đầu cộng thêm một. Lý do là phía sau xâu ban đầu được thêm một số nhỏ hơn mọi ký tự, chẳng hạn 0, chủ yếu để tiện cài đặt hàm. m là tham số so sánh của sắp xếp cơ sở; nếu xâu chỉ gồm chữ cái và chữ số thông thường theo bảng ASCII thì đặt $m = 128$ là đủ.
4. Dòng 7–10 thực hiện sắp xếp cơ sở: tách xâu thành n ký tự độ dài 1 rồi sắp xếp, cuối cùng lưu kết quả vào mảng sa . Vòng lặp `for` ở dòng 10 bắt đầu từ $n - 1$; mục đích là khi gặp các ký tự giống nhau, ký tự có vị trí trước hơn trong xâu cũng có thứ tự trước hơn.
5. Trong điều kiện vòng lặp ở dòng 11, j mỗi lần được nhân đôi, nghĩa là mỗi lượt dùng kết quả đã sắp xếp của lượt trước làm nền. Biến p có hai tác dụng: một là chỉ số tăng dần của mảng, hai là thứ hạng của xâu con đứng sau cùng sau khi sắp xếp. Nếu p bằng n , mọi xâu con đã có thứ tự khác nhau và vòng lặp kết thúc. Câu lệnh $m = p$ là một tối ưu nhỏ: trong lần sắp xếp dùng thứ hạng của lượt trước làm khóa, tham số sắp xếp lớn nhất chỉ cần là p .
6. Dòng 13–14 thực hiện sắp xếp theo khóa thứ hai. Chẳng hạn nếu độ dài của ký tự gốc là 1, khóa thứ hai chính là ký tự ở vị trí kế tiếp. Vì khóa thứ hai của j ký tự cuối là rỗng, hoặc bằng 0, chúng chắc chắn nằm ở đầu thứ tự; dòng 13 xử lý bước này. Do đã có thứ tự theo khóa thứ nhất, chỉ cần dịch trái mỗi giá trị trong mảng sa một vị trí là thu được vị trí bắt đầu của các xâu con đã sắp theo khóa thứ hai.
7. Dòng 15 lưu thứ hạng của các vị trí bắt đầu, vốn đã nhận được theo thứ tự khóa thứ hai, vào mảng wv . Mảng này được dùng trong lần sắp xếp cơ sở tiếp theo để xác định thứ tự giữa hai xâu con. Khi chỉ có một ký tự, giá trị được lưu không phải là thứ hạng mà là giá trị ứng với ký tự đó, nhưng vai trò là như nhau.
8. Dòng 16–19 tiếp tục sắp xếp các xâu con, vốn đã có thứ tự theo khóa thứ hai, theo khóa thứ nhất. Chú ý rằng mảng wv lấy giá trị từ x , vì y lưu chỉ số của các xâu con đã có thứ tự theo khóa thứ hai. Vòng lặp cuối bắt đầu từ $n - 1$, nhờ đó với hai khóa thứ nhất giống nhau, vị trí đứng sau trong xâu vẫn đứng sau trong thứ tự cuối cùng; nói cách khác, thứ tự theo khóa thứ hai vẫn được giữ ổn định.
9. Dòng 20–21 tìm thứ hạng tương ứng cho các xâu con đã có thứ tự, với thông tin thứ tự đang nằm trong mảng sa . Cần chú ý rằng hai xâu con hoàn toàn giống nhau phải nhận cùng một thứ hạng. Mảng x thu được cuối cùng lưu thông tin thứ hạng của từng xâu con.
10. Sau một số lượt lặp, thứ tự của mọi xâu con đều khác nhau, vòng lặp kết thúc. Khi đó sa là mảng hậu tố, còn x là mảng thứ hạng.

Tác giả gốc nói rằng phải tự mô phỏng theo ví dụ mới hiểu rõ thuật toán. Ví dụ trong bài dùng xâu `aabaaaaab`; khi cài đặt, ta xét thêm ký tự canh 0 ở cuối:

$$r = \{97, 97, 98, 97, 97, 97, 97, 98, 0\}, \quad n = 9.$$

Sau lượt sắp xếp từng ký tự, ta có

$$sa = [8, 0, 1, 3, 4, 5, 6, 2, 7], \quad x = [1, 1, 2, 1, 1, 1, 1, 2, 0].$$

Ở lượt $j = 1$, sau khi sắp xếp theo khóa thứ hai rồi khóa thứ nhất:

$$y = [8, 7, 0, 2, 3, 4, 5, 1, 6], \quad sa = [8, 0, 3, 4, 5, 1, 6, 7, 2],$$

và mảng thứ hạng trở thành

$$x = [1, 2, 4, 1, 1, 1, 2, 3, 0].$$

Ở lượt $j = 2$:

$$y = [7, 8, 6, 1, 2, 3, 4, 5, 0], \quad sa = [8, 3, 4, 5, 0, 6, 1, 7, 2],$$

cuối cùng

$$x = [4, 6, 8, 1, 2, 3, 5, 7, 0].$$

Vì đã có $p = n$, mọi hậu tố đã được phân biệt và thuật toán kết thúc.

2 Thuật toán DC3

Ý tưởng chính là chia toàn bộ xâu thành hai phần: các hậu tố có vị trí bắt đầu đồng dư 0 theo modulo 3, ký hiệu là A , và các hậu tố có vị trí bắt đầu không đồng dư 0 theo modulo 3, ký hiệu là B . Sau đó nối hai phần của B , tức phần đồng dư 1 và phần đồng dư 2, để tạo thành một xâu mới. Cứ mỗi 3 ký tự của xâu mới được xem như một ký tự rồi đem sắp xếp cơ sở để thu được thứ tự. Nếu vẫn tồn tại hai ký tự hoàn toàn giống nhau, ta gọi đệ quy hàm này cho đến khi mọi ký tự đều phân biệt thứ tự. Cuối cùng, dùng thứ tự của phần B làm khóa thứ hai và trộn với phần A đã được sắp xếp theo khóa thứ nhất, ta thu được mảng hậu tố cuối cùng.

```
#define F(x) ((x)/3+((x)%3==1?0:tb))
#define G(x) ((x)<tb?(x)*3+1:((x)-tb)*3+2)
int wa[maxn],wb[maxn],wv[maxn],ws[maxn];
int c0(int *r,int a,int b) {
    return r[a]==r[b]&&r[a+1]==r[b+1]&&r[a+2]==r[b+2];
}
int c12(int k,int *r,int a,int b) {
    if(k==2)
        return r[a]<r[b]||r[a]==r[b]&&c12(1,r,a+1,b+1);
    else
        return r[a]<r[b]||r[a]==r[b]&&wv[a+1]<wv[b+1];
}
void sort(int *r,int *a,int *b,int n,int m) {
    int i;
    for(i=0;i<n;i++) wv[i]=r[a[i]];
    for(i=0;i<m;i++) ws[i]=0;
    for(i=0;i<n;i++) ws[wv[i]]++;
    for(i=1;i<m;i++) ws[i]+=ws[i-1];
    for(i=n-1;i>=0;i--) b[--ws[wv[i]]]=a[i];
    return;
}
void dc3(int *r,int *sa,int n,int m)
{
    int i,j,*rn=r+n,*san=sa+n,ta=0,tb=(n+1)/3,tbc=0,p;
    r[n]=r[n+1]=0;
    for(i=0;i<n;i++) if(i%3!=0) wa[tbc++]=i;
    sort(r+2,wa,wb,tbc,m);
    sort(r+1,wb,wa,tbc,m);
    sort(r,wa,wb,tbc,m);
    for(p=1,rn[F(wb[0])]=0,i=1;i<tbc;i++)
        rn[F(wb[i])]=c0(r,wb[i-1],wb[i])?p-1:p++;
    if(p<tbc) dc3(rn,san,tbc,p);
    else for(i=0;i<tbc;i++) san[rn[i]]=i;
    for(i=0;i<tbc;i++) if(san[i]<tb) wb[ta++]=san[i]*3;
    if(n%3==1) wb[ta++]=n-1;
```

```

sort(r,wb,wa,ta,m);
for(i=0;i<tbc;i++) wv[wb[i]=G(san[i])]=i;
for(i=0,j=0,p=0;i<ta && j<tbc;p++)
sa[p]=c12(wb[j]%3,r,wa[i],wb[j])?wa[i++]:wb[j++];
for(;i<ta;p++) sa[p]=wa[i++];
for(;j<tbc;p++) sa[p]=wb[j++];
return;
}

```

1. Trong tham số ở dòng 24, hai mảng rn và san phục vụ lời gọi đệ quy. rn lưu thông tin thứ hạng của xâu hiện tại, còn san lưu mảng hậu tố của xâu hiện tại.
2. Dòng 25 đặt thêm 0 vì trước khi nối hai phần có vị trí không đồng dư 0 theo modulo 3, ta cần bảo đảm độ dài mỗi phần là bội của 3 để có thể sắp xếp cơ sở. Với độ dài bất kỳ, thêm nhiều nhất hai số 0 là đủ; ở đây làm vậy để tiện cho các bước sau.
3. Dòng 26–29 sắp xếp cơ sở các xâu con của xâu mới, mỗi xâu con gồm một nhóm 3 ký tự, cuối cùng lưu mảng hậu tố vào wb . Chú ý rằng vị trí của hai mảng trung gian wa và wb được hoán đổi một lần; việc này làm cho kết quả sắp xếp theo chữ số cao hơn đồng thời phụ thuộc vào kết quả đã sắp xếp theo chữ số thấp hơn.
4. Dòng 30–31 dùng hàm F để ánh xạ vị trí bắt đầu của hậu tố trong xâu ban đầu sang vị trí trong xâu mới. Cần nhớ rằng trong xâu mới, 3 ký tự được xem như 1 ký tự. Từ đó ta tính mảng thứ hạng rn của các ký tự trong xâu mới.
5. Dòng 32–33: nếu trong lần sắp xếp này mọi ký tự đều không hoàn toàn giống nhau, nghĩa là đã phân biệt được toàn bộ thứ tự, ta trực tiếp tính mảng hậu tố và lưu vào san . Nếu vẫn tồn tại các ký tự hoàn toàn giống nhau, ta gọi đệ quy hàm này và kết quả vẫn nằm trong san . Vì trước mỗi lần đệ quy, độ dài xâu mới không vượt quá $2/3$ độ dài xâu ban đầu, đệ quy chắc chắn dừng.
6. Dòng 34–36 dùng phần có vị trí không đồng dư 0 theo modulo 3 làm khóa thứ hai, lấy trực tiếp thông tin thứ tự từ san hiện tại. Cần chú ý trường hợp đặc biệt $n \% 3 == 1$: do không tồn tại $san[n]$, ta phải gán riêng. Sau đó dùng sắp xếp cơ sở để trộn với phần đồng dư 0 theo modulo 3, tức khóa thứ nhất, thu được mảng hậu tố của phần đồng dư 0 và lưu trong wa .
7. Dòng 37 ánh xạ vị trí từng ký tự của xâu mới về xâu ban đầu bằng hàm G , từ đó thu được mảng hậu tố wb và mảng thứ hạng wv cho các hậu tố có vị trí không đồng dư 0 theo modulo 3 của xâu ban đầu.
8. Dòng 38–41 trộn hai phần: phần đồng dư 0 và phần không đồng dư 0 theo modulo 3. Khi mọi phần tử của một trong hai mảng hậu tố wa hoặc wb đã được lấy ra, thứ tự của các phần tử còn lại trong mảng kia đều đứng sau, nên chỉ cần nối chúng vào cuối sa .

Ví dụ trong bài là xâu $aabaaaaba$. Khi thêm ký tự canh 0, ta có

$$r = [1, 1, 2, 1, 1, 1, 1, 2, 1, 0], \quad n = 10.$$

Các vị trí không đồng dư 0 theo modulo 3 ban đầu là

$$wa = [1, 2, 4, 5, 7, 8].$$

Sau khi sắp xếp ba khóa, gán thứ hạng cho xâu mới, sắp xếp phần đồng dư 0 và trộn hai phần, mảng hậu tố cuối cùng là

$$sa = [9, 8, 3, 4, 5, 0, 6, 1, 7, 2].$$

3 Mảng height

Định nghĩa ngắn gọn

Mảng height là độ dài tiền tố chung dài nhất của hai hậu tố kề nhau trong thứ tự đã sắp. Ví dụ, với hậu tố $sa[\text{rank}[i]]$ của $\text{rank}[i]$ và hậu tố $sa[\text{rank}[i] - 1]$ đứng ngay trước nó, tiền tố chung dài nhất của chúng được ghi là $\text{height}[\text{rank}[i]]$, viết tắt là $h[i]$. Nói cách khác, $h[i]$ là độ dài tiền tố chung dài nhất giữa hậu tố bắt đầu ở vị trí i trong xâu và hậu tố đứng ngay trước nó theo thứ tự từ điển.

Tính chất quan trọng

Với mọi $i \geq 1$, ta có

$$h[i] \geq h[i - 1] - 1.$$

Chúng minh: giả sử i và j thỏa

$$\text{rank}[i] - \text{rank}[j] = 1.$$

Khi đó độ dài tiền tố chung dài nhất của hai hậu tố này là $h[i]$. Bây giờ xét hai hậu tố trong xâu ban đầu có vị trí bắt đầu sau j và sau i , ký hiệu là $j + 1$ và $i + 1$.

Trường hợp thứ nhất: ký tự đầu của hậu tố i và hậu tố j khác nhau. Khi đó $h[i] = 0$, nên hiển nhiên $h[i + 1] \geq h[i] - 1$.

Trường hợp thứ hai: ký tự đầu của hậu tố i và hậu tố j giống nhau. Khi đó hậu tố $i + 1$ và hậu tố $j + 1$ lần lượt là hậu tố i và hậu tố j sau khi bỏ ký tự đầu. Rõ ràng hậu tố $j + 1$ đứng trước hậu tố $i + 1$, và tiền tố chung dài nhất của chúng có độ dài $h[i] - 1$. Trong các hậu tố đứng trước hậu tố $i + 1$ theo thứ tự sắp xếp, hậu tố kề với nó chắc chắn có tiền tố chung dài nhất với nó; có thể hiểu là độ tương tự cao nhất. Do đó hậu tố có thứ hạng

$$\text{rank}[i + 1] - 1$$

và hậu tố có thứ hạng $\text{rank}[i + 1]$ có tiền tố chung dài nhất ít nhất là $h[i] - 1$. Suy ra

$$h[i + 1] \geq h[i] - 1.$$

Lấy ví dụ xâu `aabaaaab`. Mảng rank của nó là

$$[4, 6, 8, 1, 2, 3, 5, 7].$$

Giả sử $i = 6$ và $j = 5$. Khi đó

$$\text{suffix}(i) = \text{ab}, \quad \text{suffix}(j) = \text{aab}.$$

Hậu tố thứ $i + 1$ là `b`, còn hậu tố thứ $j + 1$ là `ab`. Hậu tố $j + 1$ đứng trước hậu tố $i + 1$, đồng thời

$$\text{rank}[i + 1] = 8, \quad \text{rank}[j + 1] = 7.$$

Vì thứ hạng của hậu tố $j + 1$ vừa đúng là thứ hạng ngay trước hậu tố $i + 1$, ở ví dụ này đạt dấu bằng:

$$\text{height}[\text{rank}[7]] \geq \text{height}[\text{rank}[6]] - 1.$$

Cài đặt

```
int rank[maxn], height[maxn];
void calheight(int *r, int *sa, int n)
{
    int i, j, k=0;
    for(i=1; i<=n; i++) rank[sa[i]]=i;
    for(i=0; i<n; height[rank[i++]]=k)
        for(k?k--:0, j=sa[rank[i]-1]; r[i+k]==r[j+k]; k++);
    return;
}
```

1. Dòng 5 dùng mảng hậu tố của xâu để tính mảng thứ hạng tương ứng.
2. Dòng 6–7: khi k bằng 0, ta bắt đầu tìm lần lượt từ ký tự đầu tiên. Sau đó, theo thứ tự của mảng thứ hạng, ta lần lượt tìm giá trị h tiếp theo. Nhờ tính chất trên, lúc này không cần tìm lại từ đầu mà chỉ cần bắt đầu từ vị trí của ký tự thứ $k - 1$; cứ như vậy, ta thu được toàn bộ giá trị của mảng `height`.